

Backend Architecture

Shop-Ez Application: MVC Architectural Pattern

Architecture Flow: Users send requests to the **Controller**. The Controller processes these requests by interacting with the **Model** (which performs DB Operations on the Database). The **View** (Routes) defines the endpoints that direct to the Controller. The Controller then sends a response back to the User.

The Shop-Ez backend application follows the **Model-View-Controller (MVC)** architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The **Model** layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using **Mongoose**, which provides a schema-based solution to model application data for MongoDB.

Controller Layer

The **Controller** layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

View Layer (Routing Layer)

In the context of a backend REST API, the **View** is implemented as the **routing layer**, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

- **Separation of Concerns:** Each layer has a clearly defined responsibility, improving readability and maintainability.

- **Scalability:** New features can be added easily by creating new routes, controllers, and models.
- **Reusability:** Logic in controllers and models can be reused across multiple parts of the application.
- **Testing:** Each layer can be tested independently, especially the controllers and models.
- **Collaboration-Friendly:** Multiple developers can work simultaneously on different layers without conflict.